

Отчет по заданию 3.1

“Теория Параллелизма”

В данном задании требуется реализовать многопоточную версию программы умножения матрицы на вектор с параллельной инициализацией массивов, используя такие классы, как `std::thread`, `std::jthread` или асинхронные задания.

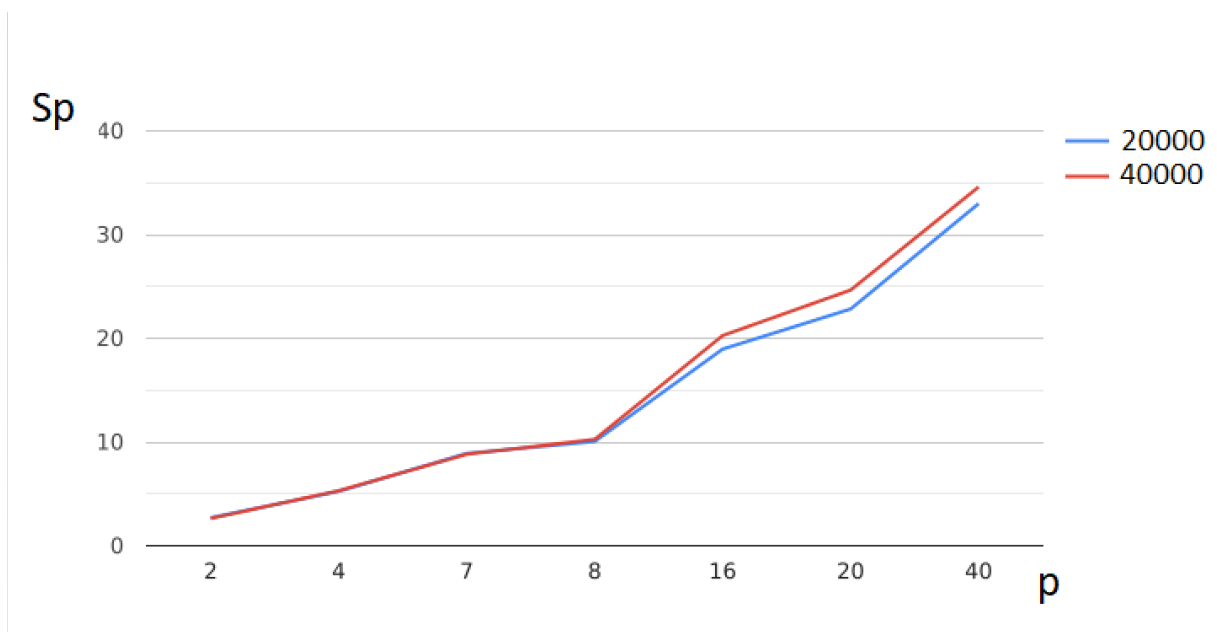
Код, предоставленный к данной работе, имеет 2 версии реализации: с использованием класса `std::thread`, и с дополнительными блокировками (`std::condition_variable`, `std::mutex` ...)

Ниже представлена таблица масштабируемости на основе первого примера:

Вариант	Количество потоков								
	2			4		7		8	
	T1	T2	S2	T4	S4	T7	S7	T8	S8
20000^2	4816	1781	2,704	918	5,246	541	8,902	479	10,054
40000^2	19130	7280	2,628	3607	5,304	2169	8,82	1866	10,252

Вариант	Количество потоков					
	16		20		40	
	T16	S16	T20	S20	T40	S40
20000^2	254	18,961	211	22,825	146	32,986
40000^2	944	20,265	776	24,652	553	34,593

В данной таблице данные были предоставлены в миллисекундах. Далее приведем график масштабируемости:



Вывод о масштабируемости:

Исходя из результатов, видно, что графики практически имеют идентичный закон по масштабированию скорости выполнения работы программы. Кроме того, скорость работы линейно возрастает с увеличением числа потоков, что говорит о хорошей оптимизации распараллеливания потоков на протяжении всей работы программы. С результатами программы `Source_mod.cpp` идет ускорение линейно на 5-6 миллисекунд из за того что мы не объединяем потоки после инициализации, а сразу идем выполнять следующий блок кода - умножение.